A Reproducibility Study of SGD, SVRG, and SAGA on Regularized Logistic Regression

Jihyun Kim and William Zhang

Abstract

This project studies three stochastic optimization methods on the same convex finite-sum problem: L2-regularized binary logistic regression on the `sklearn` breast cancer dataset. The methods are stochastic gradient descent (SGD), stochastic variance-reduced gradient (SVRG), and SAGA. The main goal is not to claim a new algorithm, but to reproduce and compare the practical behavior of these methods under one controlled setup. We evaluate convergence, wall-clock runtime, gradient-evaluation cost, stability across random seeds, and memory usage. The main question is how much benefit we get from variance reduction, and what it costs in extra computation or storage. The final results show that SVRG reached the lowest final objective in the locked run, SAGA was competitive when tuned to its best learning rate, and SGD was the cheapest method in memory and gradient work.

1 Introduction

Stochastic gradient methods are a standard tool for machine learning optimization. They are attractive because each update can be much cheaper than computing a full gradient over the whole dataset. The downside is that a single sample gradient is noisy. This noise can make the objective decrease unevenly, especially when the method gets close to a good solution.

Variance-reduced methods try to keep the low per-iteration cost of stochastic methods while making the update direction more reliable. In this project, we compare plain SGD with two variance-reduced methods, SVRG and SAGA. The study is set up as a paper reading and reproducibility project rather than a new method proposal. The goal is to implement the algorithms, run them on a clean convex problem, and understand the tradeoffs that appear in practice.

The problem we use is L2-regularized logistic regression on the breast cancer dataset from `sklearn`. This is a useful test case because the objective is convex, the dataset is small enough to inspect carefully, and the finite-sum structure matches the setting where SVRG and SAGA are usually discussed. We focus on four practical questions:

- Which method reduces the objective fastest at the beginning?
- Which method reaches the best final objective value?
- How much do the methods differ in runtime and gradient evaluations?
- How do memory cost and stability change when using variance reduction?

2 Background and Motivation

For empirical risk minimization problems, the objective often has the form

$$F(\theta) = \frac{1}{n}\sum_{i=1}^{n} f_i(\theta) + r(\theta),$$

where f_i is the loss for one training example and r is a regularization term. Full gradient descent uses all n examples for every update, which gives a stable direction but can be expensive for large datasets. SGD instead samples one example, or a mini-batch, and uses that gradient as an estimate of the full gradient.

The main motivation for SVRG and SAGA is that the stochastic gradient in SGD can have high variance. Even when the current point is near the optimum, a single sample gradient may point in a noticeably different direction from the full gradient. Because of this, SGD often needs a decreasing learning rate or it keeps fluctuating near the solution.

Variance-reduced methods use extra information to reduce this noise. SVRG uses periodic full-gradient calculations at a snapshot point. SAGA stores past per-sample gradients and uses their average as a correction. Both methods spend extra resources to get a better update direction. In convex finite-sum settings, this is the high-level reason these methods can converge faster than plain SGD when the learning rate is chosen well.

I do not prove any new theoretical results in this project. The relevant theory from the literature says, roughly, that variance-reduced methods can improve the convergence behavior for finite-sum convex problems because their gradient estimates become more accurate as the iterates approach a solution. For this report, I treat that theory as motivation and then test whether the expected tradeoffs show up in my implementation.

3 Problem Formulation

The dataset is `sklearn.datasets.load_breast_cancer`. It has 569 samples and 30 features. The features are standardized before optimization. The original binary labels are converted to

$$y_i \in \{-1, +1\}.$$

After conversion, the class counts are 212 samples with label -1 and 357 samples with label $+1$.

The model is binary logistic regression with parameters $w \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$. The objective is

$$F(w, b) = \frac{1}{n} \sum_{i=1}^{n} \log \Big(1 + \exp \Big(-y_i(w^\top x_i + b)\Big)\Big) + \frac{\lambda}{2} \|w\|_2^2.$$

The regularization parameter is

$$\lambda = 10^{-3}.$$

The bias term is not regularized.

For implementation, I store the parameters as one vector

$$\theta = (w, b) \in \mathbb{R}^{d+1}.$$

The full gradient is used for objective checking and for SVRG snapshot steps. The per-sample gradient is used by all three stochastic methods. I also checked the gradient implementation numerically using finite differences:

$$\text{maximum finite difference error} = 7.173 \times 10^{-11}.$$

This check is important because a small gradient bug would make the algorithm comparison meaningless.

4 Methods

4.1 SGD

SGD is the baseline method. At each step, it samples one index i_t uniformly from the training set and updates

$$\theta_{t+1} = \theta_t - \eta \nabla f_{i_t}(\theta_t),$$

where η is the learning rate. In this project, I use a constant learning rate for each run and tune it with a small grid search.

The main advantage of SGD is simplicity. It uses one sample gradient per update and has very low memory cost. The main disadvantage is that the update direction is noisy. In the results, I expect SGD to make fast early progress but to be less smooth near the final objective value.

4.2 SVRG

SVRG uses a snapshot parameter $\tilde{\theta}$. At the start of each outer loop, it computes the full gradient $\nabla F(\tilde{\theta})$. Then, inside the inner loop, it samples an index i_t and uses

$$v_t = \nabla f_{i_t}(\theta_t) - \nabla f_{i_t}(\tilde{\theta}) + \nabla F(\tilde{\theta}).$$

The update is

$$\theta_{t+1} = \theta_t - \eta v_t.$$

The correction term makes the stochastic update closer to the full-gradient direction while still using a sampled index in the inner loop. In my implementation, the inner loop length is 569, which is exactly one pass over the dataset. The tradeoff is that SVRG does not store a large table of gradients, but it does pay for repeated full-gradient computations.

4.3 SAGA

SAGA keeps a table of stored gradients, one for each training example. At each step, it samples an index, computes the current per-sample gradient for that index, and combines it with the stored table average. A simple way to write the update direction is

$$v_t = \nabla f_{i_t}(\theta_t) - \alpha_{i_t} + \frac{1}{n} \sum_{j=1}^{n} \alpha_j,$$

where α_j is the stored gradient for sample j. After the update, the table entry for i_t is replaced by the new gradient.

In my implementation, the table stores the unregularized per-sample logistic loss gradients, and the L2 regularization gradient is added using the current w. This keeps the stored table from depending on an old value of the regularization term.

SAGA's main cost is memory. It stores an $n \times (d+1)$ gradient table, so the storage grows with both the number of examples and the number of features. On the breast cancer dataset this is small, but the scaling is important when thinking about larger problems.

5 Experimental Setup

All methods use the same objective, preprocessing, initialization, and stopping condition. The initial parameter vector is the zero vector. Each run uses 30 epochs. I record the objective value after each epoch, along with the cumulative number of gradient evaluations and the wall-clock runtime.

The learning-rate grids are:

$$\text{SGD: } \{0.001, 0.003, 0.01, 0.03, 0.1\},$$

$$\text{SVRG: } \{0.005, 0.01, 0.03, 0.05, 0.1, 0.2\},$$

$$\text{SAGA: } \{0.005, 0.01, 0.03, 0.05, 0.1, 0.2\}.$$

After the sweep, I choose one learning rate per method for the locked comparison:

$$\eta_{\text{SGD}} = 0.03, \quad \eta_{\text{SVRG}} = 0.1, \quad \eta_{\text{SAGA}} = 0.1.$$

The main metrics are:

- final objective value,
- objective value versus gradient evaluations,
- objective value versus runtime,
- sensitivity to learning rate,
- stability across seeds,
- memory usage of the optimizer state.

The saved seed experiment uses seeds 0, 1, and 2. I use this experiment to check whether one method is consistently better or whether the result is mainly due to one lucky random run. The current seed CSV contains SGD and SVRG, so I only report seed statistics for those two methods.

6 Numerical Results

6.1 Learning-Rate Sweep

Before comparing the methods directly, I first tune the learning rate for each method. Table 1 reports the best learning rate from the sweep for each method. The full sweep is shown visually in Figure 1 and is also saved in `outputs/lr_sweep_summary.csv`.

Table 1: Best learning rates from the sweep with 30 epochs and seed 0.

Method	Learning rate	Final loss	Gradient evals	Runtime (s)
SVRG	0.100	0.0598532343	51210	0.3574
SGD	0.030	0.0641138605	17070	0.2065
SAGA	0.030	0.0603328688	17639	0.2579

The sweep shows that all three methods need a reasonable learning rate. For SGD, 0.03 gave the best final loss, while 0.1 was already too aggressive. For SVRG, 0.1 gave the best final loss, but 0.2 made the result worse. For SAGA, the best swept learning rate was 0.03, not the locked value

0.1. This matters because SAGA at 0.03 reached a final loss close to SVRG while using many fewer gradient evaluations.

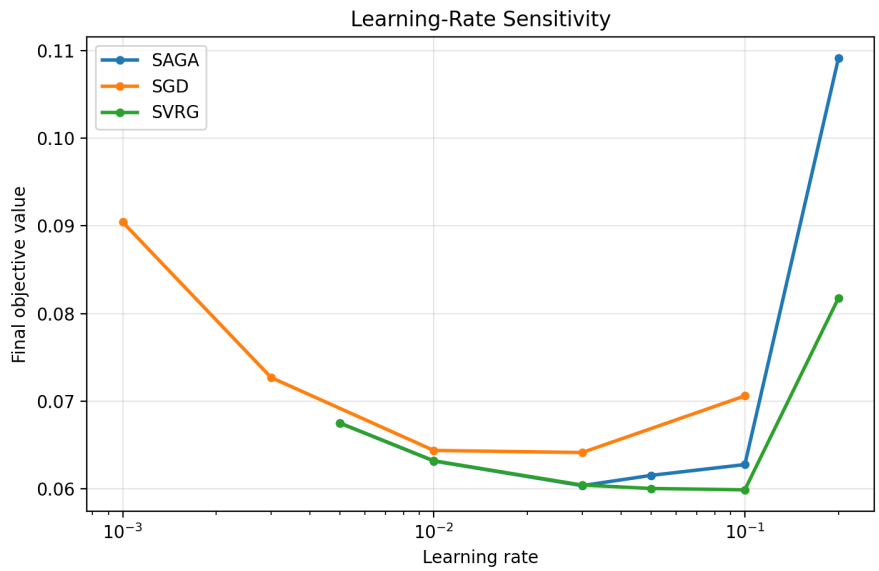

Figure 1: Learning-rate sensitivity for SGD, SVRG, and SAGA.

Figure 1 gives a visual check of the sweep. This figure is useful because it shows whether each method has a wide stable range or only works well for a narrow set of learning rates. In this experiment, the variance-reduced methods improved a lot when moving from very small learning rates to moderate ones, but both SVRG and SAGA became worse at 0.2. This was the clearest sign that the step size still matters even when variance reduction is used.

6.2 Main Comparison

After choosing the learning rates, I compare the three methods in one locked run. Table 2 includes the final objective, runtime, and gradient-evaluation count for each method.

Table 2: Main locked comparison with seed 0.

Method	Learning rate	Final loss	Gradient evals	Runtime (s)
SGD	0.03	0.0641138605	17070	0.1774
SVRG	0.10	0.0598532343	51210	0.3122
SAGA	0.10	0.0627392703	17639	0.2262

This table is the main numerical summary of the project. It shows the basic tradeoff between objective quality, runtime, and gradient-evaluation cost. SVRG reached the lowest final objective value, but it used about three times as many gradient evaluations as SGD. SGD had the fastest runtime in this locked run. SAGA used almost the same gradient-evaluation budget as SGD, after its initial table setup, and ended between SGD and SVRG at the locked learning rate.

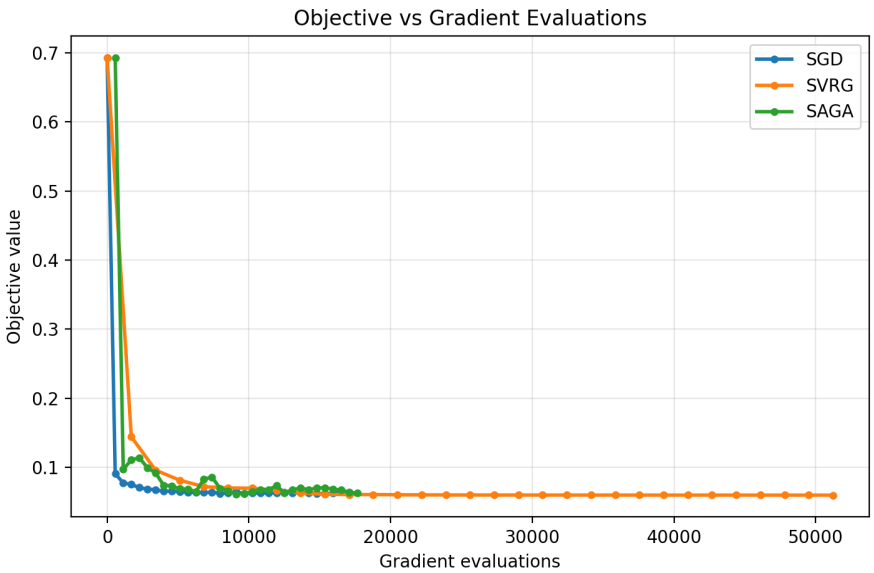

Figure 2: Objective value versus gradient evaluations.

Figure 2 compares the methods by gradient-evaluation cost rather than by time. This is important because the methods do different amounts of gradient work per epoch. SGD gets a large drop very quickly, but then its curve is noisy and it settles at a higher objective. SVRG decreases more smoothly and reaches the best final value, but it spends many more gradient evaluations. SAGA has a fast initial drop, but at the locked step size its curve is less smooth than SVRG's.

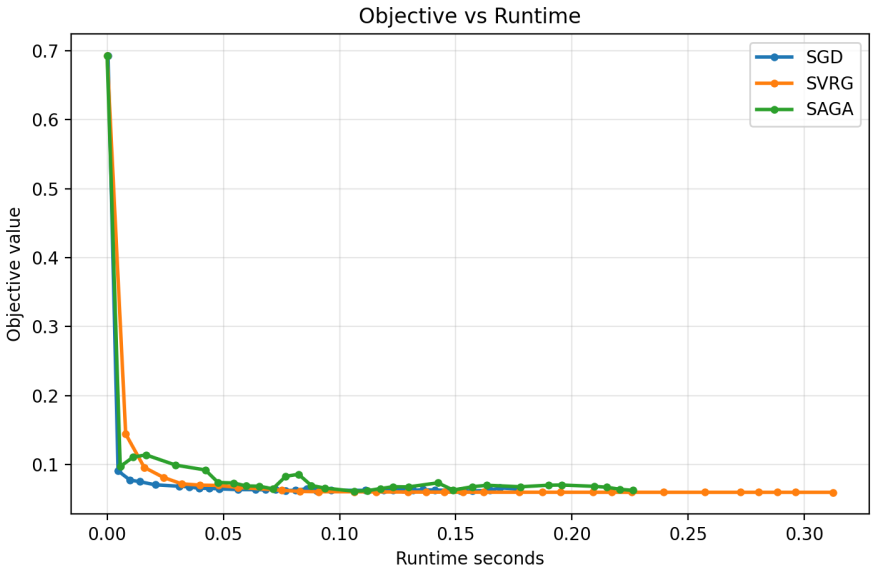

Figure 3: Objective value versus wall-clock runtime.

Figure 3 compares the methods using actual runtime. This is the most practical view, but it also depends on implementation details and hardware. SGD was fastest in total time for the locked run. SVRG took longer because of the repeated full-gradient computations, but it also reached the best final objective. SAGA was between the two in runtime. The runtime plot matches the main

tradeoff from the gradient-evaluation plot: variance reduction improves the final objective, but the improvement comes with extra work.

6.3 Stability Across Seeds

Table 3 reports the saved three-seed summary for the locked configuration. The file contains SGD and SVRG results for seeds 0, 1, and 2.

Table 3: Seed stability summary from seeds 0, 1, and 2.

Method	Mean final loss	Std. final loss	Mean runtime (s)	Mean grad evals
SGD	0.0625934339	0.0013355237	0.3879	17070
SVRG	0.0598533388	0.0000094993	0.6513	51210

For the two reported methods, SVRG had a much smaller final-loss standard deviation than SGD. I do not report a SAGA seed statistic because it is not in the saved seed summary file.

6.4 Memory and Compute Tradeoff

The dataset itself is shared by all methods, so the memory comparison in Table 4 focuses on optimizer state.

Table 4: Memory and compute tradeoff summary.

Method	Extra memory	Extra compute pattern	Practical comment
SGD	about 0.5 KB	one sample gradient per step	lowest memory, noisy final behavior
SVRG	about 1.2 KB	periodic full gradients	best final loss, highest grad evals
SAGA	about 138.5 KB	stored gradient table	good tuned loss, largest memory

For this dataset, SAGA's memory use is still small in absolute terms. The important point is scaling: SAGA grows with $n(d+1)$, while SGD and SVRG use only small $O(d)$ optimizer state.

7 Discussion / Take-Home Messages

Variance reduction helped, but the benefit was not free.

- **Fastest early progress:** SGD and SAGA made the largest early drops; SGD was especially cheap in gradient evaluations.
- **Best final objective:** SVRG had the best locked final loss, 0.0598532343. SAGA's best swept result was close at 0.0603328688.
- **Most stable method:** For saved seed results, SVRG was most stable: 9.4993×10^{-6} final-loss standard deviation versus 1.3355×10^{-3} for SGD.
- **Lowest memory cost:** SGD used the least memory. SAGA used the most because of its gradient table.
- **Practical choice:** SGD is best for simplicity, SVRG for the lowest objective when full-gradient passes are affordable, and SAGA when the gradient table fits in memory and the step size is tuned.

The main limitation is that this is one small convex dataset, so the results are a controlled tradeoff study rather than a universal benchmark.

8 Conclusion

SVRG gave the best final objective value, SGD was the cheapest and fastest locked baseline, and SAGA became competitive when its learning rate was tuned carefully. The practical lesson is that variance reduction can help on this convex finite-sum problem, but it trades off against extra gradient work for SVRG or extra memory for SAGA.

References

[1] A. Defazio, F. Bach, and S. Lacoste-Julien. SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives. 2014.
[2] R. Johnson and T. Zhang. Accelerating Stochastic Gradient Descent using Predictive Variance Reduction. 2013.
[3] H. Robbins and S. Monro. A Stochastic Approximation Method. 1951.
[4] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization Methods for Large-Scale Machine Learning. 2018.